



End-to-End Kubernetes Deployments: From Single-Tier to Multi-Tier Applications

Task 1: Deploy Super Mario & Pacman with NodePort Services

Objective :

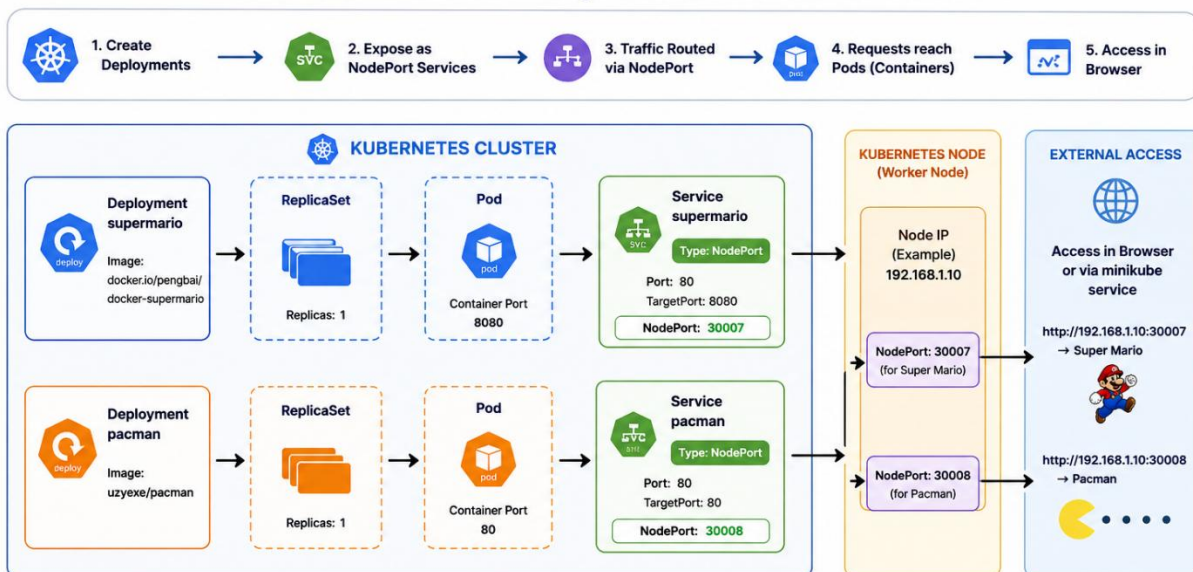
Create two deployments using the following container images and expose them externally using **NodePort services**:

- [docker.io/pengbai/docker-supermario](https://hub.docker.com/r/pengbai/docker-supermario/)
- [uzyexe/pacman](https://hub.docker.com/r/uzyexe/pacman/)

Prerequisites :

- Kubernetes cluster (Minikube / Kubeadm / OpenShift / Kind)
- kubectl configured and working

Kubernetes Architecture – Super Mario & Pacman with NodePort



⚙️ Step 1: Create Deployment for Super Mario

```
root@controlplane:~$ kubectl create deployment supermario --image=docker.io/pengbai/docker-supermario
deployment.apps/supermario created
```

🔍 Verify Deployment:

```
root@controlplane:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
supermario-dc9c695d4-k57ld         1/1     Running   0           60s
```

⚙️ Step 2: Expose Super Mario Deployment as NodePort

```
root@controlplane:~$ kubectl expose deployment supermario --type=NodePort --port=80 --target-port=8080
service/supermario exposed
```

Check Service:

```
root@controlplane:~$ kubectl get svc
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes      ClusterIP   10.96.0.1    <none>        443/TCP          5d17h
supermario      NodePort    10.106.146.6 <none>        80:30151/TCP     14s
```

⚙️ Step 3: Create Deployment for Pacman

```
root@controlplane:~$ kubectl create deployment pacman --image=uzyexe/pacman
deployment.apps/pacman created
```

🔍 Verify Deployment:

```
root@controlplane:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
pacman-6db7c86b98-c5qfh            1/1     Running   0           11s
supermario-dc9c695d4-k57ld         1/1     Running   0           2m33s
```

⚙️ Step 4: Expose Pacman Deployment as NodePort

```
root@controlplane:~$ kubectl expose deployment pacman --type=NodePort --port=80 --target-port=80
service/pacman exposed
```

🔍 Check Service:

```
root@controlplane:~$ kubectl get svc
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes      ClusterIP   10.96.0.1    <none>        443/TCP          5d17h
pacman          NodePort    10.97.223.69 <none>        80:30313/TCP     12s
supermario      NodePort    10.106.146.6 <none>        80:30151/TCP     2m39s
```

🌐 Step 5: Access Port Number on the browser



HTML5 PACMAN

[Writeup](#) | [Code on Github](#)



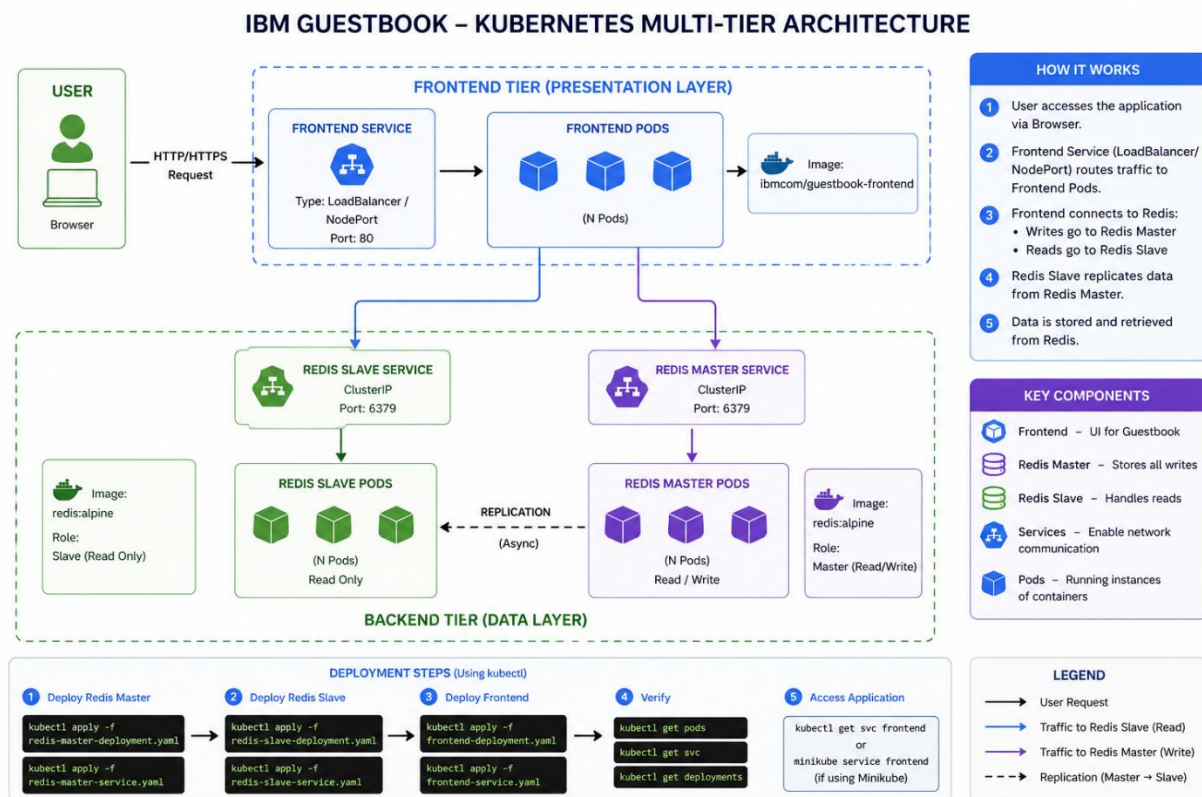
🚀 Task 2: Kubernetes Multi-Tier Deployment (Guestbook App)

📌 Architecture Overview

The Guestbook app is a 3-tier application:

1. Frontend (Web UI) → User interacts
2. Redis Master (Database) → Stores data
3. Redis Slave (Replica DB) → Reads data

👉 This is a classic multi-tier architecture used in real DevOps projects



Step 1: Clone Repository

```
root@controlplane:~$ git clone https://github.com/IBM/guestbook.git
Cloning into 'guestbook'...
remote: Enumerating objects: 448, done.
remote: Total 448 (delta 0), reused 0 (delta 0), pack-reused 448 (from 1)
Receiving objects: 100% (448/448), 205.42 KiB | 4.78 MiB/s, done.
Resolving deltas: 100% (264/264), done.
```

Step 2: Deploy guestbook-deployment

```
root@controlplane:~/guestbook/v1$ kubectl apply -f guestbook-deployment.yaml
deployment.apps/guestbook-v1 created
```

Step 3: Deploy guestbook-service

```
root@controlplane:~/guestbook/v1$ kubectl apply -f guestbook-service.yaml
service/guestbook created
```

Verify Deployment:

```
root@controlplane:~/guestbook/v1$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
guestbook-v1-85675f5766-drhd5      1/1     Running   0           26s
guestbook-v1-85675f5766-nv7w7      1/1     Running   0           26s
guestbook-v1-85675f5766-pxb9m      1/1     Running   0           26s
```

Check Service:

```
root@controlplane:~/guestbook/v1$ kubectl get svc
NAME      TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
guestbook LoadBalancer  10.105.133.38 <pending>    3000:30943/TCP   22s
kubernetes ClusterIP   10.96.0.1     <none>       443/TCP         6d12h
```

Step 4: Access Port Number on the browser

Guestbook - v1

<https://f5362eba8777-10-244-8-214-30943.spca.r.killercoda.com/>
[/env](#) [/info](#)

Task 3: Kubernetes Single-Tier Deployment (ParksMap App)

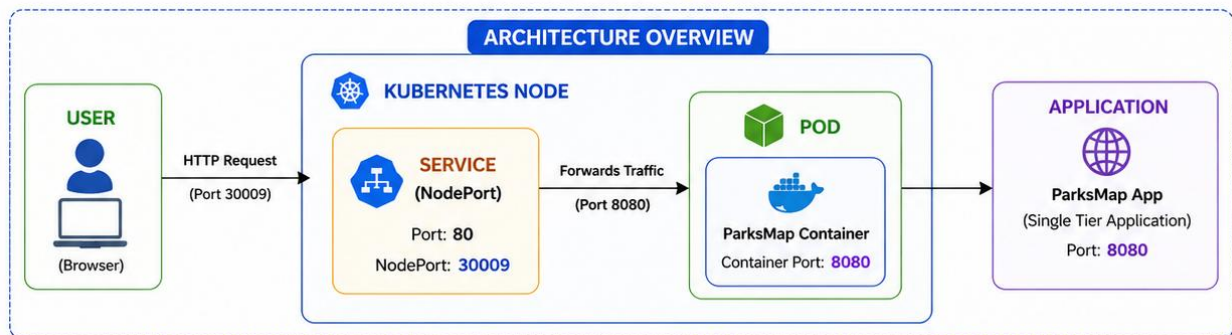
Objective

Deploy a single-tier application using the image:

- quay.io/openshiftroadshow/parksmap
- Application runs internally on port 8080

KUBERNETES – SINGLE TIER APPLICATION DEPLOYMENT

 **Objective:** Deploy ParksMap application using image quay.io/openshiftroadshow/parksmap (Application runs on port 8080 internally)



DEPLOYMENT STEPS

- Create Deployment**

```
kubectl create deployment parksmap \
--image=quay.io/openshiftroadshow/parksmap
```
- Expose as NodePort Service**

```
kubectl expose deployment parksmap \
--type=NodePort \
--port=80 \
--target-port=8080
```
- Verify Deployment**

```
kubectl get pods
kubectl get svc
kubectl get deployment
```
- Access Application**

```
http://<NODE-IP>:<NODE-PORT>
```

If using Minikube:
`minikube service parksmap`

RESOURCE SUMMARY

Resource	Name	Type	Details
	parksmap	Manages Pods and ReplicaSet	Image: quay.io/openshiftroadshow/parksmap
	parksmap	NodePort	Port: 80 TargetPort: 8080 NodePort: 30000–32767
	parksmap-xxx	Running Application	Container Port: 8080

EXAMPLE OUTPUT

kubectl get pods					kubectl get svc parksmap				
NAME	READY	STATUS	RESTARTS	AGE	NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
parksmap-7f6d9f8c8b-x2k9n	1/1	Running	0	2m	parksmap	NodePort	10.96.145.32	80:30000/TCP	2m

TRAFFIC FLOW

```
graph TD
    User[User opens browser] --> Service[Request hits NodePort Service  
(Port: 30009)]
    Service --> Pod[Service forwards traffic to Pod  
(TargetPort 8080)]
    Pod --> App[ParksMap application responds to user]
```

The traffic flow diagram shows the sequence of events: a user opens a browser, a request hits the NodePort service (port 30009), the service forwards traffic to the pod (target port 8080), and the ParksMap application responds to the user.

KEY POINTS

- ✓ Single tier application – no database or backend dependency.
- ✓ Application runs on port 8080 inside the container.
- ✓ Service of type NodePort exposes the application on each node.
- ✓ Access using <NodeIP>:<NodePort> in a browser.

USEFUL COMMANDS

<code>kubectl get all</code>	– List all resources
<code>kubectl describe pod <pod-name></code>	– Describe pod details
<code>kubectl describe svc parksmap</code>	– Describe service details
<code>kubectl logs <pod-name></code>	– View application logs
<code>kubectl delete deployment parksmap</code>	– Delete deployment
<code>kubectl delete svc parksmap</code>	– Delete service

CLEAN UP

```
kubectl delete deployment parksmap
kubectl delete svc parksmap
```


⚙️ Step 1: Create Deployment

```
root@controlplane:~$ kubectl create deployment parksmap --image=quay.io/openshiftroadshow/parksmap
deployment.apps/parksmap created
```

⚙️ Step 2: Expose as NodePort Service

```
root@controlplane:~$ kubectl expose deployment parksmap --type=NodePort --port=80 --target-port=8080
service/parksmap exposed
```

🔍 Verify Deployment

```
root@controlplane:~$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
parksmap-6cdb7d5c49-ksrkm          1/1     Running   0           45s
```

🔍 Check Service:

```
root@controlplane:~$ kubectl get svc
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes  ClusterIP   10.96.0.1     <none>       443/TCP          6d13h
parksmap   NodePort    10.108.212.58 <none>       80:30614/TCP     19s
```

🌐 Step 3: Access Port Number on the browser



🚀 Task 4: Kubernetes Multi-Tier Deployment (Knote App)

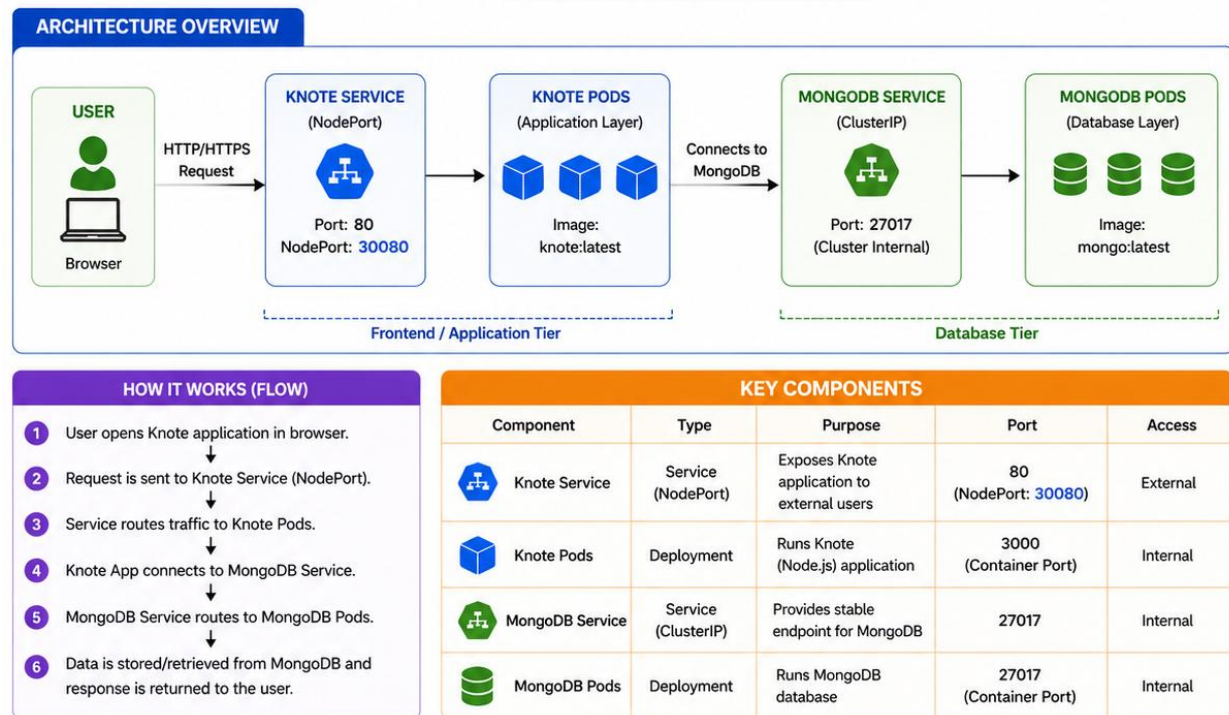
📌 Objective

Deploy the Knote application (multi-tier app) from:

👉 <https://github.com/ashutoshbhakare/knote>

KNOTE – MULTI TIER APPLICATION DEPLOYMENT ON KUBERNETES

Repository: <https://github.com/ashutoshbhakare/knote>



⚙️ Step 1: Clone Repository

```
root@controlplane:~$ git clone https://github.com/ashutoshbhakare/knote.git
Cloning into 'knote'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 18 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (18/18), 4.35 KiB | 4.35 MiB/s, done.
Resolving deltas: 100% (5/5), done.
```

⚙️ Step 2: Deploy knotedeployment

```
root@controlplane:~/knote$ kubectl apply -f knotedeployment.yml
deployment.apps/knote created
```

⚙️ Step 2: Deploy knotesvc

```
root@controlplane:~/knote$ kubectl apply -f knotesvc.yml
service/knote created
```

🔍 Verify Deployment:

```
root@controlplane:~/knote$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
knote-75bdd596d5-fblbd             1/1     Running   0           27s
parksmap-6cdb7d5c49-ksrkm          1/1     Running   0           20m
```

🔍 Check Service:

```
root@controlplane:~/knote$ kubectl get svc
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
knote     NodePort    10.106.6.117  <none>       80:32076/TCP     24s
kubernetes ClusterIP  10.96.0.1     <none>       443/TCP        6d13h
parksmap  NodePort    10.108.212.58 <none>       80:30614/TCP     19m
```

🌐 Step 3: Access Port Number on the browser

knote

A simple note-taking app.

Upload an image

Docker-Logo...768x432.png

Write your content here

You don't have any notes yet.